Министерство образования Республики Беларусь

Учреждение образования

«Белорусский государственный университет информатики и радиоэлектроники»

Кафедра электронных вычислительных машин

Лабораторная работа №4
«Программирование часов реального времени»

Выполнил:
Студент группы 450502
Новицкий Е.Ю.

Проверил:
Преподаватель
Марзалюк А.В.

Минск, 2026

1. Постановка задачи

Написать программу, которая будет считывать и устанавливать время в часах реального времени. Считанное время должно выводиться на экран в удобочитаемой форме.

1. Используя аппаратное прерывание часов реального времени и режим генерации периодических прерываний реализовать функцию задержки с точностью в миллисекунды.
2. Используя аппаратное прерывания часов реального времени и режим будильника реализовать функции программируемого будильника.

2. Алгоритм

Перед установкой значений времени вызывается функция, которая считывает и анализирует старший байт регистра состояния 1 на предмет доступности значений для чтения и записи. Когда этот бит установлен в '0', отключается внутренний цикл обновления часов реального времени: для этого старший бит регистра состояния 2 устанавливается в '1'.

Считывание или запись значений времени происходит следующим образом: в порт 70h отправляется индекс регистра CMOS, соответствующий значению времени (секунды, часы и т. д.), затем происходит чтение значения из порта 71h (или запись значения в порт).

После установки значений времени вызывается функция, которая возобновляет внутренний цикл обновления часов реального времени.

Для реализации функции задержки заменён обработчик прерывания 0x70, в котором происходит отсчёт миллисекунд. Для включения периодического прерывания, происходящего примерно каждую миллисекунду, 6-й бит регистра B устанавливается в '1'.

3. Листинг программы

Далее приведен листинг программы, реализующей все поставленные задачи.

```
#include <stdio.h>
#include <dos.h>
#include <conio.h>

#define RTC_INDEX_PORT 0x70 // choose
#define RTC_DATA_PORT  0x71 // read or write

// RTC registers
#define RTC_SECONDS      0x00
#define RTC_ALARM_SEC    0x01
#define RTC_MINUTES      0x02
#define RTC_ALARM_MIN    0x03
```

```
#define RTC_HOURS        0x04
#define RTC_ALARM_HOUR   0x05
#define RTC_WEEKDAY      0x06
#define RTC_DAY          0x07
#define RTC_MONTH        0x08
#define RTC_YEAR         0x09
#define RTC_STATUS_A     0x0A
#define RTC_STATUS_B     0x0B
#define RTC_STATUS_C     0x0C
#define RTC_STATUS_D     0x0D
#define RTC_CENTURY      0x32


// PIC
// rtc on irq8 – second pic
#define PIC1_COMMAND     0x20
#define PIC1_DATA        0x21
#define PIC2_COMMAND     0xA0
#define PIC2_DATA        0xA1
#define PIC_EOI          0x20


volatile unsigned long rtc_ticks = 0; // tick counter
volatile unsigned long target_ticks = 0; // ticks to wait
volatile int delay_done = 0; // flag of delay end
volatile int alarm_fired = 0; // flag of alarm worked

// old_handler
void interrupt (*old_rtc_handler)(void);

// old pic masks, to recover after
unsigned char old_pic1_mask = 0;
unsigned char old_pic2_mask = 0;


unsigned char read_rtc(unsigned char reg)
{
   outp(RTC_INDEX_PORT, reg);
   return inp(RTC_DATA_PORT);
}
```

```
void write_rtc(unsigned char reg, unsigned char value)
{
   outp(RTC_INDEX_PORT, reg);
   outp(RTC_DATA_PORT, value);
}


int bcd_to_int(unsigned char bcd)
{
   return ((bcd >> 4) * 10) + (bcd & 0x0F);
}


unsigned char int_to_bcd(int value)
{
   return (unsigned char)(((value / 10) << 4) | (value % 10));
}


void wait_rtc_ready(void)
{
   while (read_rtc(RTC_STATUS_A) & 0x80) {}
}

int rtc_is_binary_mode(void)
{
   unsigned char regB = read_rtc(RTC_STATUS_B);
   return (regB & 0x04) ? 1: 0;
}

int rtc_is_24hour_mode(void)
{
   unsigned char regB = read_rtc(RTC_STATUS_B);
   return (regB & 0x02) ? 1: 0;
}

// read and set time

void show_current_time(void)
{
```

```
unsigned char sec, min, hour, day, month, year, century;
int isBinary;

wait_rtc_ready();

sec = read_rtc(RTC_SECONDS);
min = read_rtc(RTC_MINUTES);
hour = read_rtc(RTC_HOURS);
day = read_rtc(RTC_DAY);
month = read_rtc(RTC_MONTH);
year = read_rtc(RTC_YEAR);
century = read_rtc(RTC_CENTURY);

isBinary = rtc_is_binary_mode();

if (!isBinary)
{
   sec = bcd_to_int(sec);
   min = bcd_to_int(min);
   hour = bcd_to_int(hour);
   day = bcd_to_int(day);
   month = bcd_to_int(month);
   year = bcd_to_int(year);
   century = bcd_to_int(century);

}

clrscr();
printf("==========================================\n");
printf("      CURRENT RTC TIME AND DATE\n");
printf("==========================================\n\n");

printf("Date : %02u.%02u.%02u%02u\n", day, month, century, year);
printf("Time : %02u:%02u:%02u\n", hour, min, sec);

printf("\nStorage format: %s\n", isBinary ? "binary" : "BCD");
printf("Hour mode     : %s\n", rtc_is_24hour_mode() ? "24-hour" : "12-hour");

printf("\nPress any key...");
getch();
```

```
}


int validate_datetime(int hour, int min, int sec, int day, int month, int year, int cen-
tury)
{
   if (hour < 0 || hour > 23) return 0;
   if (min < 0 || min > 59) return 0;
   if (sec < 0 || sec > 59) return 0;
   if (day < 1 || day > 31) return 0;
   if (month < 1 || month > 12) return 0;
   if (year < 0 || year > 99) return 0;
   if (century < 0 || century > 99) return 0;

   return 1;

}


void set_current_time(void)
{
   // check UIP
   // set bit 7 regB
   // write new values
   // reset bit 7 reB

   int hour, min, sec, day, month, year, century;
   unsigned char regB;
   int isBinary;

   clrscr();
   printf("=========================================\n");
   printf("       SET RTC TIME AND DATE\n");
   printf("=========================================\n\n");

   printf("Enter hours      (0-23): ");
   scanf("%d", &hour);

   printf("Enter minutes    (0-59): ");
   scanf("%d", &min);
```

```
printf("Enter seconds    (0-59): ");
scanf("%d", &sec);

printf("Enter day        (1-31): ");
scanf("%d", &day);

printf("Enter month      (1-12): ");
scanf("%d", &month);

printf("Enter year       (0-99): ");
scanf("%d", &year);

printf("Enter century    (e.g. 20 for 2026): ");
scanf("%d", &century);

if (!validate_datetime(hour, min, sec, day, month, year, century))
{
   printf("\nError: invalid input data.");
   printf("\nPress any key...");
   getch();
   return;
}


wait_rtc_ready();


regB = read_rtc(RTC_STATUS_B);

// set bit 7
write_rtc(RTC_STATUS_B, regB | 0x80);

isBinary = rtc_is_binary_mode();

if (!isBinary)
{
   write_rtc(RTC_SECONDS, int_to_bcd(sec));
   write_rtc(RTC_MINUTES, int_to_bcd(min));
   write_rtc(RTC_HOURS,   int_to_bcd(hour));
   write_rtc(RTC_DAY,     int_to_bcd(day));
   write_rtc(RTC_MONTH,   int_to_bcd(month));
```

```
        write_rtc(RTC_YEAR,    int_to_bcd(year));
        write_rtc(RTC_CENTURY, int_to_bcd(century));
    }
    else
    {
        write_rtc(RTC_SECONDS, (unsigned char)sec);
        write_rtc(RTC_MINUTES, (unsigned char)min);
        write_rtc(RTC_HOURS,   (unsigned char)hour);
        write_rtc(RTC_DAY,     (unsigned char)day);
        write_rtc(RTC_MONTH,   (unsigned char)month);
        write_rtc(RTC_YEAR,    (unsigned char)year);
        write_rtc(RTC_CENTURY, (unsigned char)century);
    }


    write_rtc(RTC_STATUS_B, regB & 0x7F);

    printf("\nNew time has been set successfully.");
    printf("\nPress any key...");
    getch();
}


// rtc interrupt

unsigned int rate_to_frequency(unsigned char rate)
{
    switch (rate)
    {
        case 0x06: return 1024;
        case 0x07: return 512;
        case 0x08: return 256;
        case 0x09: return 128;
        case 0x0A: return 64;
        default:   return 1024;
    }
}


void set_periodic_rate(unsigned char rate)
{
```

```
    // rtc periodic frequency set-up
    // lower 4 bits regA = RS3..RS0

    unsigned char regA;

    wait_rtc_ready();

    regA = read_rtc(RTC_STATUS_A);

    regA = (regA & 0xF0) | (rate & 0x0F);

    write_rtc(RTC_STATUS_A, regA);

}


void enable_periodic_interrupt(void)
{
    // enable periodic interrupt rtc
    // bit 6 rebB = PIE

    unsigned char regB = read_rtc(RTC_STATUS_B);
    write_rtc(RTC_STATUS_B, regB | 0x40);
}


void disable_periodic_interrupt(void)
{
    unsigned char regB = read_rtc(RTC_STATUS_B);
    write_rtc(RTC_STATUS_B, regB & 0xBF);
}


void enable_alarm_interrupt(void)
{
    // enable alarm rtc interrupt
    // bit 5 regB = AIE

    unsigned char regB = read_rtc(RTC_STATUS_B);
    write_rtc(RTC_STATUS_B, regB | 0x20);
}
```

```
void disable_alarm_interrupt(void)
{
   unsigned char regB = read_rtc(RTC_STATUS_B);
   write_rtc(RTC_STATUS_B, regB & 0xDF);
}


void unmask_rtc_irq(void)
{
   // allow irq8 on 2nd pic and irq2 on 1st pic
   // otherwise rtc interrupts do not reach program

   old_pic1_mask = inp(PIC1_DATA);
   old_pic2_mask = inp(PIC2_DATA);

   outp(PIC1_DATA, old_pic1_mask & (~0x04));
   outp(PIC2_DATA, old_pic2_mask & (~0x01));
}



void restore_pic_masks(void)
{
   // recover old pic masks

   outp(PIC1_DATA, old_pic1_mask);
   outp(PIC2_DATA, old_pic2_mask);
}


void clear_rtc_interrupt_flags(void)
{
   // clearing a possible hanging RTC interrupt state
   read_rtc(RTC_STATUS_C);
}


void interrupt new_rtc_handler(void)
{
```

```
    unsigned char regC;

    regC = read_rtc(RTC_STATUS_C); // confirm rtc interrupt

    if (regC & 0x40) // bit 6 = periodic interrupt
    {
        rtc_ticks++;

        if (rtc_ticks >= target_ticks)
        {
            delay_done = 1;
        }

    }

    if (regC & 0x20) // bit 5 = alarm
    {
        alarm_fired = 1;
    }

    // finish handling irq8:
    // firstly 2nd pic, after 1st

    outp(PIC2_COMMAND, PIC_EOI);
    outp(PIC1_COMMAND, PIC_EOI);

}



void install_rtc_handler(void)
{
    // set on handler int 70h

    old_rtc_handler = getvect(0x70);
    setvect(0x70, new_rtc_handler);
}


void restore_rtc_handler(void)
{
```

```
    setvect(0x70, old_rtc_handler);
}


void rtc_delay_ms(unsigned long delay_ms, unsigned char rate)
{
    // delay func in miliseconds
    // delay_ms - time to wait
    // rate - frequency code rtc

    unsigned int frequency;
    unsigned long ticks_needed;

    frequency = rate_to_frequency(rate);

    ticks_needed = (delay_ms * frequency) / 1000UL;

    if (ticks_needed == 0) ticks_needed = 1;

    rtc_ticks = 0;
    target_ticks = ticks_needed;
    delay_done = 0;

    clear_rtc_interrupt_flags();
    set_periodic_rate(rate);
    enable_periodic_interrupt();

    // wait till handler set flag
    while (!delay_done) {};

    disable_periodic_interrupt();

}



void test_delay(void)
{
    unsigned long ms;
    int rate_choice;
    unsigned char rate = 0x06;
```

```
    clrscr();
    printf("=========================================\n");
    printf("     DELAY VIA PERIODIC INTERRUPTS\n");
    printf("=========================================\n\n");

    printf("Enter delay in milliseconds: ");
    scanf("%lu", &ms);

    printf("\nSelect RTC interrupt frequency:\n");
    printf("1. 1024 Hz (06h)\n");
    printf("2.  512 Hz (07h)\n");
    printf("3.  256 Hz (08h)\n");
    printf("4.  128 Hz (09h)\n");
    printf("5.   64 Hz (0Ah)\n");
    printf("\nYour choice: ");
    scanf("%d", &rate_choice);

    switch (rate_choice)
    {
       case 1: rate = 0x06; break;
       case 2: rate = 0x07; break;
       case 3: rate = 0x08; break;
       case 4: rate = 0x09; break;
       case 5: rate = 0x0A; break;
       default: rate = 0x06; break;
    }

    printf("\nWaiting...\n");

    rtc_delay_ms(ms, rate);

    printf("Delay completed.");
    printf("\nPress any key...");
    getch();
}



// alarm
```

```
int validate_alarm_time(int hour, int min, int sec)
{
   if (hour < 0 || hour > 23) return 0;
   if (min < 0 || min > 59) return 0;
   if (sec < 0 || sec > 59) return 0;
   return 1;
}


void set_alarm_time(int hour, int min, int sec)
{
   int isBinary;

   wait_rtc_ready();
   isBinary = rtc_is_binary_mode();

   if (!isBinary)
   {
      write_rtc(RTC_ALARM_SEC,  int_to_bcd(sec));
      write_rtc(RTC_ALARM_MIN,  int_to_bcd(min));
      write_rtc(RTC_ALARM_HOUR, int_to_bcd(hour));
   }
   else
   {
      write_rtc(RTC_ALARM_SEC,  (unsigned char)sec);
      write_rtc(RTC_ALARM_MIN,  (unsigned char)min);
      write_rtc(RTC_ALARM_HOUR, (unsigned char)hour);
   }
}



void test_alarm(void)
{
   int hour, min, sec;

   clrscr();
   printf("==========================================\n");
   printf("         PROGRAMMABLE ALARM\n");
   printf("==========================================\n\n");
```

```
    printf("Enter alarm hour   (0-23): ");
    scanf("%d", &hour);

    printf("Enter alarm minute (0-59): ");
    scanf("%d", &min);

    printf("Enter alarm second (0-59): ");
    scanf("%d", &sec);

    if (!validate_alarm_time(hour, min, sec))
    {
        printf("\nError: invalid time entered.");
        printf("\nPress any key...");
        getch();
        return;
    }

    alarm_fired = 0;

    set_alarm_time(hour, min, sec);
    clear_rtc_interrupt_flags();
    enable_alarm_interrupt();

    printf("\nAlarm set to %02d:%02d:%02d", hour, min, sec);
    printf("\nWaiting for alarm...\n");

    while (!alarm_fired) {}; // wait alarm interrupt

    disable_alarm_interrupt();

    printf("\n*** ALARM TRIGGERED! ***");
    printf("\nPress any key...");
    getch();
}


void show_menu(void)
{
    clrscr();
    printf("==========================================\n");
    printf("      LABORATORY WORK #4 - RTC\n");
```

```
    printf("==========================================\n\n");
    printf("1. Read current time and date\n");
    printf("2. Set new time and date\n");
    printf("3. Delay via RTC periodic interrupts\n");
    printf("4. Programmable RTC alarm\n");
    printf("0. Exit\n\n");
    printf("Your choice: ");
}



void main(void)
{
    int choice;

    // set self handler and enable irq rtc
    install_rtc_handler();
    unmask_rtc_irq();
    clear_rtc_interrupt_flags();

    do
    {
        show_menu();
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                show_current_time();
                break;

            case 2:
                set_current_time();
                break;

            case 3:
                test_delay();
                break;

            case 4:
                test_alarm();
```

```
            break;

        case 0:
            break;

        default:
            printf("\nInvalid menu option.");
            printf("\nPress any key...");
            getch();
            break;
    }

  } while (choice != 0);

  // disable rtc modes before exit
  disable_periodic_interrupt();
  disable_alarm_interrupt();
  clear_rtc_interrupt_flags();

  // restore old handler and old masks pic
  restore_rtc_handler();
  restore_pic_masks();
}
```

4. Тестирование программы

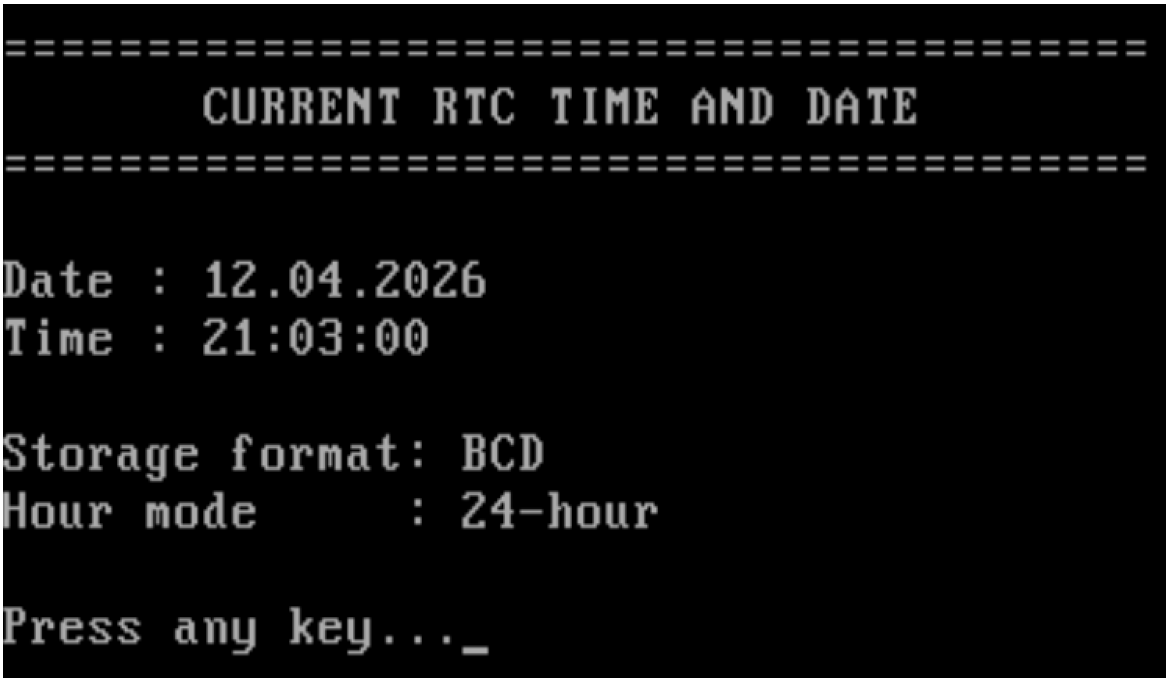

Рисунок 4.1 — Вывод текущего времени.

```
=========================================
        SET RTC TIME AND DATE
=========================================

Enter hours       (0-23): 12
Enter minutes     (0-59): 32
Enter seconds     (0-59): 16
Enter day         (1-31): 25
Enter month       (1-12): 12
Enter year        (0-99): 85
Enter century     (e.g. 20 for 2025): 19

New time has been set successfully.
Press any key...
```

Рисунок 4.2 — Установка нового времени

```
=========================================
       CURRENT RTC TIME AND DATE
=========================================

Date : 25.12.1985
Time : 12:32:21

Storage format: BCD
Hour mode     : 24-hour

Press any key...
```

Рисунок 4.3 — Вывод нового времени.

```
==========================================
          PROGRAMMABLE ALARM
==========================================

Enter alarm hour   (0-23): 12
Enter alarm minute (0-59): 40
Enter alarm second (0-59): 2

Alarm set to 12:40:02
Waiting for alarm...

*** ALARM TRIGGERED! ***
Press any key...
```

Рисунок 4.4 — Установка и срабатывание будильника.

5. Заключение

В данной лабораторной работе были выполнены все поставленные задачи: написана программа, которая считывает и устанавливает время в часах реального времени, была реализована функция программируемого будильника, используя аппаратное прерывания часов реального времени и режим будильника.

Программа компилировалась в Turbo C++ и запускалась в DOSBox-X